

Motor de Negociação Financeira e Análise de Performance de Estruturas Lineares

Simulador de Livro de Ofertas (Order Book) com Alocação Dinâmica de Nós

Grupo 5 · Estrutura de Dados · Semestre 2026

Eduardo Affonso Boide Santos – RA: 16862544 · Fernando Lacerda Dantas – RA: 17097341 · Felipe Alirio Baruja – RA: 15636442 · Nicolas Gonçalves Follone – RA: 16827586 · Victor Hugo Aparecido Galho Trasancos – RA: 16839040

📄 **Script do Apresentador:** "Bom dia a todos, bom dia professor. Nós somos o Grupo 5 e hoje vamos apresentar o desenvolvimento e a análise de performance de um simulador de livro de ofertas financeiro. O foco principal deste projeto foi construir do zero, sem o auxílio de estruturas nativas do Python, todas as estruturas de dados necessárias para processar transações na velocidade das bolsas de valores, comparando a teoria da complexidade de algoritmos com resultados empíricos coletados em testes de carga."

O Problema e as Restrições Técnicas

O motor de negociação de uma bolsa precisa gerenciar ordens de compra e venda com prioridade de preço e tempo. O desafio pedagógico foi implementar esse fluxo dinâmico **sem usar arrays ou dicionários nativos do Python**, gerenciando manualmente o religamento de ponteiros de memória em alocação dinâmica de nós, garantindo ausência de vazamentos e previsibilidade matemática.

Objetivo Central

Gerenciar a entrada de ordens e realizar o casamento (match) de preços compatíveis de ativos financeiros em tempo real.

Restrição Acadêmica Rígida

Proibido o uso de `list`, `dict`, `collections.deque` ou qualquer coleção pronta do Python no motor de negociação.

Requisitos de Baixo Nível

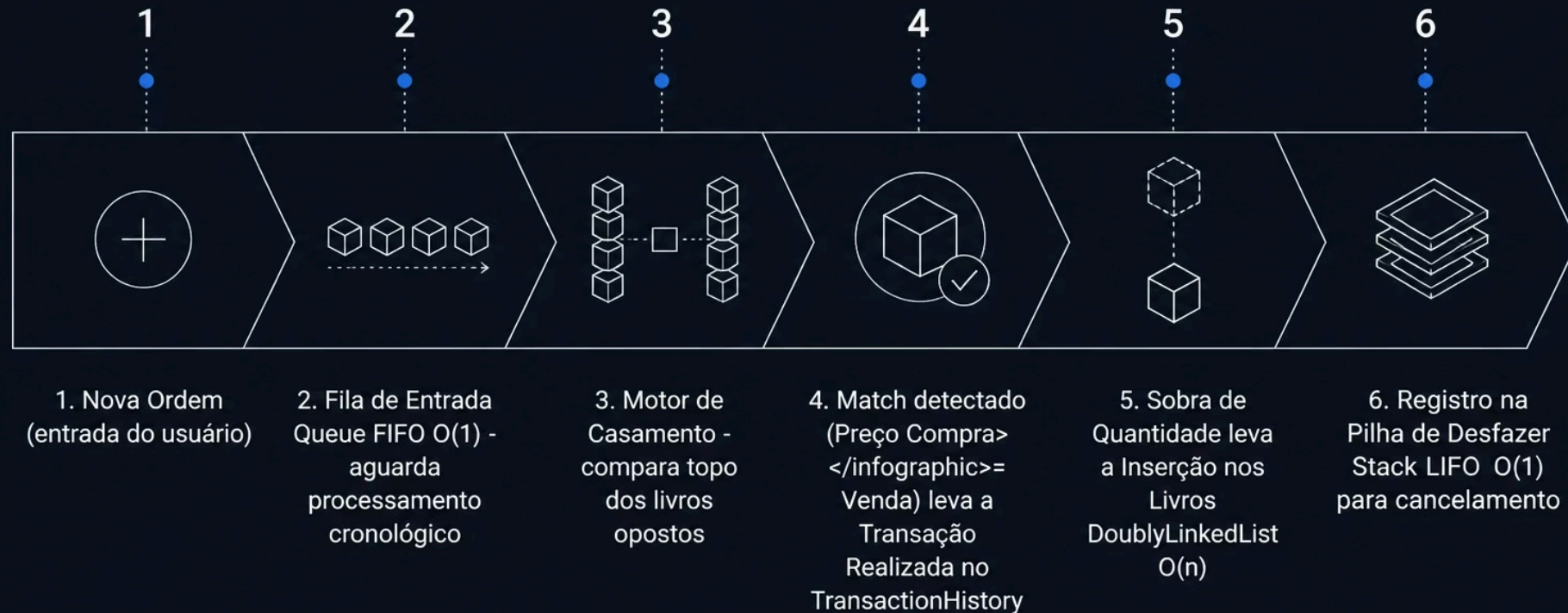
Implementação física com ponteiros (`next`, `prev`, `head`, `tail`) para simular referências de memória e alocação dinâmica de nós.

📄 **Script do Apresentador:** "O motor de negociação de uma bolsa precisa gerenciar ordens de compra e venda com prioridade de preço e tempo. O nosso desafio pedagógico foi implementar todo esse fluxo dinâmico sem usar arrays ou dicionários nativos. Tivemos que gerenciar manualmente o religamento de ponteiros de memória em alocação dinâmica de nós, garantindo que o programa não cause vazamentos de memória (nós órfãos) e seja altamente previsível matematicamente."

Arquitetura e Fluxo de Dados do Matching Engine

O diagrama abaixo representa o pipeline completo de processamento de ordens, desde a entrada na fila até o registro histórico ou a inserção nos livros de ofertas. Cada etapa possui complexidade assintótica bem definida, permitindo análise precisa de gargalos.

O fluxo garante que cada ordem seja processada em ordem cronológica estrita, com decisões de match ou inserção tomadas em tempo determinístico.



Estrutura Base e Livros de Ofertas

Classe Order

Atributos: ID, Tipo ('C'/'V'), Preço, Quantidade e Timestamp.

Classe DoublyLinkedList

Duplamente encadeada com nós dinâmicos (Node), gerenciando ponteiros head e tail para acesso direto às extremidades.

Inserção Ordenada — $O(n)$

- **Livro de Compras:** ordenação **decrecente** de preço — maior comprador na cabeça da lista
- **Livro de Vendas:** ordenação **crecente** de preço — melhor vendedor na cabeça da lista
- **Prioridade Temporal (FIFO):** operadores de desigualdade estrita garantem que, em preços idênticos, a ordem mais antiga permaneça na frente

📄 **Script do Apresentador:** "Na modelagem do livro de ofertas desenvolvida pelo Eduardo, implementamos uma Lista Duplamente Encadeada baseada na classe Node. O principal desafio foi a inserção ordenada com complexidade linear $O(n)$. A lista de compras ordena decrescentemente pelo preço, e a de vendas, crescentemente. Usamos desigualdades estritas para garantir que ordens com o mesmo preço respeitem a prioridade de quem chegou primeiro ao mercado, inserindo-as sempre atrás das mais antigas."

Fila e Pilha em Tempo Constante $O(1)$

As estruturas auxiliares garantem agilidade nas pontas do sistema, operando com complexidade assintótica constante em todas as suas operações fundamentais.

Fila de Entrada — Queue FIFO

Comportamento **First-In, First-Out**. Operações **enqueue** e **dequeue** em $O(1)$ usando ponteiros diretos para **início** e **fim**, sem necessidade de percorrer a estrutura.

Pilha de Undo — Stack LIFO

Comportamento **Last-In, First-Out**. Operações **push** e **pop** no ponteiro **topo** em $O(1)$, permitindo o cancelamento imediato da última ação executada no motor.

- ❏ **Script do Apresentador:** "A parte desenvolvida pelo Fernando garante a agilidade nas pontas do sistema. A Fila de Entrada do Simulador e a Pilha de Undo usam alocação de nós simples com ponteiros únicos. A Fila mantém referências diretas para o primeiro (início) e último nó (fim), permitindo que a inserção e remoção ocorram instantaneamente em $O(1)$, sem necessidade de percorrer a estrutura. A Pilha de Desfazer opera com complexidade constante $O(1)$, permitindo o cancelamento imediato da última ação."

Transações e Histórico Encadeado

Classe Transaction

Contratos gerados no momento do match, contendo: ID Compra, ID Venda, Preço Praticado, Quantidade e Timestamp. Cada instância representa um negócio financeiro efetivamente fechado.

TransactionHistory

Lista encadeada simples com classe dedicada NodeTransaction. Inserção no final em **O(1)** e exibição linear **O(n)** de todas as negociações consolidadas no terminal.

TRANSACTION LOG (TransactionHistory)

```
-----  
[19:12:04] MATCH! Compra #101 x Venda #102 | Qtd: 50 | R$ 45.45  
[19:12:08] MATCH! Compra #103 x Venda #104 | Qtd: 200 | R$ 120.00  
[19:12:15] MATCH! Compra #105 x Venda #106 | Qtd: 80 | R$ 350.10  
[19:12:22] MATCH! Compra #107 x Venda #108 | Qtd: 15 | R$ 98.00
```



Script do Apresentador: "Na modelagem financeira do Nicolas, toda transação representa um negócio fechado. Para manter a conformidade com as restrições do projeto, criamos a classe TransactionHistory como uma lista encadeada simples. Quando ocorre um match, o motor gera um nó de transação e o anexa ao final do histórico em O(1), preservando a integridade cronológica de toda a atividade do mercado."

O Motor de Casamento e Algoritmo de Undo

1

Condição de Execução

Preço de Compra $\geq$ Preço de Venda nos topos das listas opostas. O preço da execução é definido pela ordem que já estava esperando no livro — o **Liquidity Maker**.

2

Match Parcial vs. Total

Abatimento progressivo de quantidade nos nós dos livros. Se houver sobra, o saldo permanece no livro; se esgotado, o nó é removido com reconexão de ponteiros.

3

Algoritmo de Undo

Desempilha o ID da Stack, realiza busca linear no livro de ofertas e reconecta cirurgicamente os ponteiros `prev` e `next` dos nós vizinhos, eliminando o nó alvo sem vazamentos.

- ❏ **Script do Apresentador:** "O motor de match desenvolvido pelo Victor integra todas as estruturas. Sempre que uma ordem é processada da fila, comparamos o topo do livro oposto. Se a condição de preço for atendida, geramos a transação utilizando o preço do formador de mercado (quem já estava no livro). Caso ocorra liquidação parcial, o saldo continua no livro. Se o usuário solicitar um Undo, retiramos o ID do topo da pilha de desfazer e realizamos a remoção cirúrgica do nó no livro, reconectando os ponteiros anterior e próximo."

Menu Interativo e Inspeção de Fila em Tempo Real

Interface no Terminal

Menu estruturado com tratamento rigoroso contra entradas inválidas. Sanitização de floats e conversão segura de strings para IDs, preços e quantidades, garantindo estabilidade do sistema.

Ferramenta `mostrar_fila_entrada()`

Diferencial técnico que permite visualizar o estado da Fila de Entrada **antes** do Match, demonstrando na prática o comportamento FIFO e o controle de memória em tempo real.

```
>_ simulador_livro_ofertas.py x + v - □ x

=====
SIMULADOR DE LIVRO DE OFERTAS (ED 2026)
=====

1 - Inserir ordem de compra
2 - Inserir ordem de venda
3 - Mostrar fila de entrada (FIFO)
4 - Processar próxima ordem da fila
5 - Processar todas as ordens da fila

Opção: 3

<<<< FILA DE ENTRADA (Aguardando Processamento) >>>>
ID: 101 | Tipo: Compra | Preço: R$ 45.50 | Qtd: 100
ID: 102 | Tipo: Venda | Preço: R$ 45.45 | Qtd: 50
```



Script do Apresentador: "A parte do Felipe unificou o simulador sob um menu interativo no terminal. Criamos mecanismos robustos de validação de dados para garantir que IDs, preços e quantidades sejam lidos corretamente sem quebrar o sistema. Adicionalmente, implementamos um diferencial técnico importante: a opção de visualizar o conteúdo da fila de entrada em tempo real. Isso permite que o professor veja as ordens enfileiradas antes que o motor de match seja acionado, evidenciando o comportamento FIFO em tempo real."

Performance Empírica: Resultados dos Testes de Carga

Metodologia: geração de lotes estressantes de **1.000**, **5.000** e **10.000** ordens de compra e venda intercaladas, com medição de tempo via `time.perf_counter()` no Jupyter Notebook.

$O(1)$

Fila de Entrada

Crescimento linear estável por operação — escala perfeitamente com o volume de ordens.

$O(n^2)$

Processamento Total

Curva quadrática acumulada nos livros — valida a teoria assintótica da inserção ordenada.

$O(n)$

Gargalo Identificado

Busca linear na Lista Encadeada Ordenada é o principal limitador de performance do sistema.

📄 **Script do Apresentador:** "Para concluir, analisamos o desempenho prático do nosso simulador. Como mostram os dados e os gráficos pré-renderizados no nosso Jupyter Notebook, a Fila de Entrada escala de forma perfeitamente linear, enquanto o processamento total escala de forma quadrática. Esse comportamento valida a teoria assintótica: o gargalo do sistema é a inserção ordenada e a busca da lista encadeada, que operam em $O(n)$. Para um motor comercial real de alta frequência, a solução seria migrar dos livros lineares para árvores binárias de busca balanceadas ou heaps, diminuindo o tempo de inserção para $O(\log n)$. Agradecemos a atenção de todos e estamos abertos a perguntas!"

Conclusões e Evolução Futura

O projeto cumpriu todos os requisitos acadêmicos com rigor técnico: estruturas implementadas do zero, complexidades validadas empiricamente e motor funcional com undo, histórico e inspeção em tempo real.



Requisitos Cumpridos

Zero estruturas nativas do Python no motor.
Ponteiros gerenciados manualmente. Ausência de vazamentos de memória comprovada em testes de carga.



Teoria Validada

Resultados empíricos confirmam $O(1)$ na fila/pilha e $O(n^2)$ acumulado nos livros, alinhando prática com a análise assintótica formal.



Roadmap de Evolução

Substituir listas lineares por **Árvores AVL / Rubro-Negras** ou **Min-Max Heaps**, reduzindo a inserção para **$O(\log n)$** em ambientes de alta frequência.

📄 **Script do Apresentador:** "Em síntese, o projeto demonstrou que é possível construir um motor de negociação funcional e matematicamente previsível usando exclusivamente estruturas de dados implementadas do zero. A análise de performance validou empiricamente as complexidades teóricas, e o roadmap de evolução aponta claramente para a adoção de estruturas logarítmicas em cenários de alta frequência. Agradecemos a atenção e estamos à disposição para perguntas técnicas."